

Leçon A. Introduction au bouclage linéarisant

Exercice 1. Manivelle

1) Give the state equations of the robot. We will take the state vector $x = (x_1, x_2)$.

Une fermeture cinématique nous permet de déduire la sortie y par:

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} l_1 \cos(x_1) + l_2 \cos(x_1 + x_2) \\ l_1 \sin(x_1) + l_2 \sin(x_1 + x_2) \end{pmatrix}$$

La manivelle sera commandée par la vitesse d'un moteur à chaque articulation. On note ce vecteur de vitesse angulaire u , et on a donc:

$$u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix}$$

L'équation d'état du robot est donc:

$$\begin{cases} \dot{x}_1 = u_1 \\ \dot{x}_2 = u_2 \\ y_1 = l_1 \cos(x_1) + l_2 \cos(x_1 + x_2) \\ y_2 = l_1 \sin(x_1) + l_2 \sin(x_1 + x_2) \end{cases}$$

Le code représentant le modèle de cette manivelle est le suivant:

```
In [1]: from roblib import *

l1,l2 = 4,3
c = np.array([[3],[4]])
r=1
dt = 0.05

x = np.array([[0],[0]])
y = l1*np.array([[np.cos(x[0,0])],[np.sin(x[0,0])]]) + l2*np.array([[np.cos(x[0,
```

Pour pouvoir tracer le coude de la manivelle, on introduit le point intermédiaire z défini par:

$$z = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} l_1 \cos(x_1) \\ l_1 \sin(x_1) \end{pmatrix} \text{ et donc}$$

$$y = z + \begin{pmatrix} l_2 \cos(x_1 + x_2) \\ l_2 \sin(x_1 + x_2) \end{pmatrix}$$

On réécrit donc le code :

```
In [2]: # On prend les angles x en radian suivant:
x = np.array([[ -1],[ 2]])
```

```

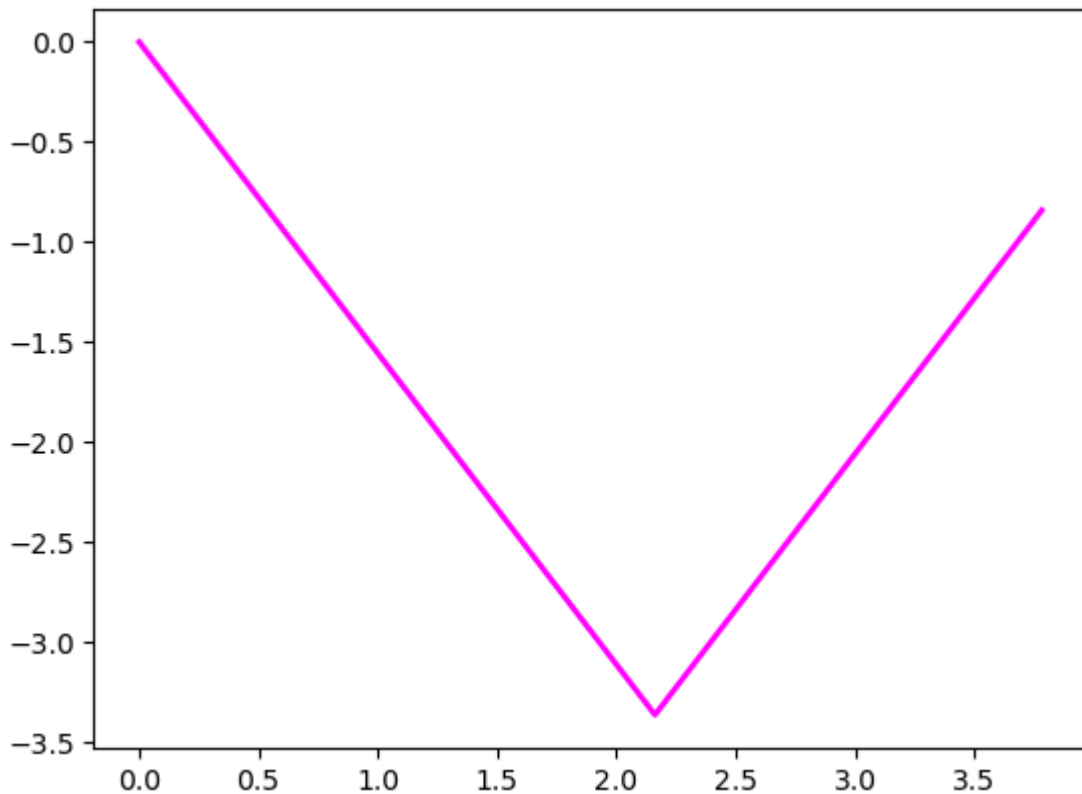
z = l1*np.array([[np.cos(x[0,0])],[np.sin(x[0,0])]])
y = z + l2*np.array([[np.cos(x[0,0]+x[1,0])],[np.sin(x[0,0]+x[1,0])]])

```

```

In [3]: # On trace un exemple de configuration de la manivelle:
plot( [0,z[0,0],y[0,0]],[0,z[1,0],y[1,0]], 'magenta', linewidth = 2)
show()

```



Si l'on désire contrôler la manivelle avec la commande en vitesse des moteurs, on peut ici créer les fonctions $z(x)$ et $y(x)$, avec x calculé à partir de l'intégrale de la commande u , calculé sur un pas de temps numérique dt :

```

In [ ]: def z(x):
        return l1*np.array([[np.cos(x[0,0])],[np.sin(x[0,0])]])

def y(x):
    return z(x) + l2*np.array([[np.cos(x[0,0]+x[1,0])],[np.sin(x[0,0]+x[1,0])]])

def draw_manivelle(x):
    plot( [0,z(x)[0,0],y(x)[0,0]],[0,z(x)[1,0],y(x)[1,0]], 'magenta', linewidth =
    # draw_disk(ax,c,r,"cyan")
    draw_disk(ax, c.flatten(), r, "cyan")

```

Pour une commande en vitesse de 1 et 2 radians par secondes par exemple , on aura:

```

In [1]: ## %matplotlib widget
# x = np.array([[ -1],[2]]) # position initiale
# dt = 0.05

# ax = plt.figure().gca()
# ax.set_xlim(0, 10); ax.set_ylim(0, 10)
# ax.set_autoscale_on(False)
# ax.grid(True)

```

```

# # line1, = ax.plot([], [], 'b+', linewidth=2.0)

# fig = plt.figure()

# for t in np.arange(0,1,0.2):
#     # clear(ax)
#     u = np.array([[1],[2]]) # commande
#     x = x + u*dt
#     fig.canvas.draw()
#     draw_manivelle(x)
#     pause(0.01)
# # ioff()

# show()

```

Le code final a été refait dans un script à part : 01_manivelle.py pour répondre aux problèmes d'animation sur jupyter notebook.

On désire déterminer la commande u de sorte à ce que le point y suive la trajectoire du cercle défini par:

$$w = c + r \cdot \begin{pmatrix} \cos(t) \\ \sin(t) \end{pmatrix}$$

Pour ce faire, on va chercher à linéariser le modèle, en dérivant un certain nombre de fois la sortie y jusqu'à retomber sur la commande u :

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} l_1 \cos(x_1) + l_2 \cos(x_1 + x_2) \\ l_1 \sin(x_1) + l_2 \sin(x_1 + x_2) \end{pmatrix}$$

$$\dot{y} = \begin{pmatrix} \dot{y}_1 \\ \dot{y}_2 \end{pmatrix} = \begin{pmatrix} -l_1 \dot{x}_1 \sin(x_1) - l_2 (\dot{x}_1 + \dot{x}_2) \sin(x_1 + x_2) \\ l_1 \dot{x}_1 \cos(x_1) + l_2 (\dot{x}_1 + \dot{x}_2) \cos(x_1 + x_2) \end{pmatrix}$$

$$\dot{y} = \begin{pmatrix} -(l_1 \sin(x_1) + l_2 \sin(x_1 + x_2)) \dot{x}_1 - l_2 \dot{x}_2 \sin(x_1 + x_2) \\ (l_1 \cos(x_1) + l_2 \cos(x_1 + x_2)) \dot{x}_1 + l_2 \dot{x}_2 \cos(x_1 + x_2) \end{pmatrix}$$

d'où

$$\dot{y} = \begin{pmatrix} -(l_1 \sin(x_1) + l_2 \sin(x_1 + x_2)) & -l_2 \sin(x_1 + x_2) \\ (l_1 \cos(x_1) + l_2 \cos(x_1 + x_2)) & l_2 \cos(x_1 + x_2) \end{pmatrix} \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix}$$

$$\text{avec } u = \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix}$$

On note $A(x)$ la matrice à gauche de u , et on choisit $u = \text{inv}(A(x)) \cdot v$ ce qui nous donne alors $\dot{y} = v$

On a réalisé une seule dérivation, et il n'y a pas de perturbation a priori. On choisit donc un régulateur de type PD de sorte que :

$$v = \alpha_1 \cdot (w - y) + \dot{w}$$

Avec α_1 un paramètre à déterminer selon le placement de pôles choisi.

En définissant l'erreur entre la sortie et la consigne par $e = (y - w)$, on obtient l'équation différentielle suivante: $\alpha_1 \cdot e + \dot{e} = 0$

Le polynôme caractéristique de cette équation est $P(s) = \alpha_1 + s$

Un placement en -1 comme demandé dans l'énoncé revient à avoir les racines de se polynôme en -1, ce qui se traduit ici par $\alpha_1 = 1$

$$\text{D'où } v = \begin{pmatrix} c+r(\cos(t)-\sin(t))-l_1.\cos(x_1)-l_2.\cos(x_1+x_2) \\ c+r(\sin(t)+\cos(t))-l_1.\sin(x_1)-l_2.\sin(x_1+x_2) \end{pmatrix}$$

3) Study the singularities of the control.

Pour s'assurer que la commande que l'on souhaite réaliser est possible, il faut pouvoir inverser la matrice $A(x)$. Les points où cette matrice ne l'est pas sont les points de singularité. Pour savoir si elle est inversible, on va calculer son déterminant:

$$\det(A) = \det \begin{pmatrix} -(l_1.\sin(x_1) + l_2.\sin(x_1 + x_2)) & -l_2.\sin(x_1 + x_2) \\ (l_1.\cos(x_1) + l_2.\cos(x_1 + x_2)) & l_2.\cos(x_1 + x_2) \end{pmatrix}$$

det est linéaire par rapport à chaque colonne:

$$\begin{aligned} \det(A) &= \det \begin{pmatrix} -l_1.\sin(x_1) & -l_2.\sin(x_1 + x_2) \\ l_1.\cos(x_1) & l_2.\cos(x_1 + x_2) \end{pmatrix} + \det \begin{pmatrix} -l_2.\sin(x_1 + x_2) & -l_2.\sin(x_1 + x_2) \\ l_2.\cos(x_1 + x_2) & l_2.\cos(x_1 + x_2) \end{pmatrix} \\ &= l_1 l_2 \det \begin{pmatrix} -\sin(x_1) & -\sin(x_1 + x_2) \\ \cos(x_1) & \cos(x_1 + x_2) \end{pmatrix} + l_2^2 \det \begin{pmatrix} -\sin(x_1 + x_2) & -\sin(x_1 + x_2) \\ \cos(x_1 + x_2) & \cos(x_1 + x_2) \end{pmatrix} \end{aligned}$$

Le terme de droite est nul. Il nous reste:

$$\det(A) = l_1 l_2 \det \begin{pmatrix} -\sin(x_1) & -\sin(x_1 + x_2) \\ \cos(x_1) & \cos(x_1 + x_2) \end{pmatrix} = l_1 l_2 (\sin(x_1 + x_2) \cos(x_1) - \cos(x_1 + x_2) \sin(x_1))$$

On reconnait en trigonométrie la formule de $\sin(a - b) = \sin(a)\cos(b) - \cos(a)\sin(b)$, d'où:

$$\det(A) = l_1 l_2 \sin(x_2)$$

Les points de singularités sont déterminés par:

$$\det(A) = 0 \iff \sin(x_2) = 0 \iff x_2 = k.\pi \text{ avec } k \in \mathbb{Z}$$

Concrètement, cela se produit lorsque les 2 bras sont soit totalement étendus et alignés, soit alignés avec le second bras juxtaposé sur le premier. Le système n'est alors plus contrôlable avec notre commande sur ces deux points, mais la dynamique du système nous permet de ne pas s'en occuper.

On pourrait le vérifier en commandant le bras de sorte à ce qu'il trace le cercle de centre la jonction des 2 bras et de rayon le bras l_2 .

Ou un autre test ou le bras n'a pas d'autre choix que d'être aligné pour suivre la trajectoire du cercle.

Dans les 2 cas, on constate sur l'animation que le bras devient instable en passant par un de ces points singuliers, et le mouvement est saccadé durant un durans un cours

instant. Il arrive que la simulation donne des choses aberrantes avec le bras qui passe d'un côté à l'autre sur une grande étendue. Le fait d'avoir un $\det(A) = 0$ fait passer la commande vers une valeur infinie, qui est difficilement compréhensible en simulation.

In []: